

计算机视觉课程设计报告

基于 21 关键点 MLP 的 实时手势控制系统

算法原理 · 架构演化 · 工程实践

项目定位 浏览器内使用 MediaPipe 提取 21 个手部关键点, 经几何归一化后送入自训的 17K 参数 MLP, 完成 10 类静态手势识别; 再结合时序平滑、动态轨迹分析与几何 reranker, 提高交互稳定性, 并通过 WebSocket 与 Python 后端联动实现系统级键鼠控制。

课程名称	计算机视觉课程设计
报告类型	技术实践报告
项目主题	实时手势识别与跨显示协议控制系统
姓 名	徐子锐
学 号	
班 级	
指导教师	
完成日期	2026 年 5 月 12 日

关键词: MediaPipe; 手部关键点; MLP; Otsu; Sobel; 时序平滑; evdev; Cloudflare Tunnel

目录

1	绪论：从一个失败的起点说起	1
1.1	从端到端 CNN 到关键点 + MLP 的范式切换	1
1.2	本报告结构	1
2	系统流水线全景	2
2.1	五阶段流水线	2
2.2	数据流形态	3
3	计算机视觉工作量详述	4
3.1	阶段一：图像预处理与可视化	4
3.1.1	灰度化：BT.601 加权	4
3.1.2	Otsu 自适应二值化	5
3.1.3	Sobel 边缘检测	6
3.1.4	为什么 UI 要支持视角切换	6
3.2	阶段三：几何特征工程	6
3.2.1	三步归一化	6
3.2.2	用单元测试硬验证	7
3.2.3	跨语言一致性：Python ↔ JS	7
3.3	阶段五：时序后处理（详见 §6）	8
4	为什么不直接用 CNN 做端到端分类？	8
4.1	当前路线本身就是“CNN（特征提取）+ MLP（分类）”	8
4.2	为什么不自己再训一个端到端 CNN	9
4.3	这一选择是否符合 CV 课程定位？	9
5	MLP 模型设计、训练与数据采集	10
5.1	模型架构	10
5.1.1	参数量精确计算	11

5.1.2	为什么是 3 层、为什么是 128/64?	11
5.2	训练配置	12
5.3	数据采集流程	12
5.4	训练曲线与最终指标	13
6	运行时三层后处理	14
6.1	层 1: 时间平滑 (消抖)	14
6.1.1	算法核心	14
6.1.2	代价与收益	15
6.2	层 2: 动态手势检测 (轨迹分析)	15
6.2.1	算法	15
6.2.2	为什么是 400 ms / 0.22 / 700 ms?	16
6.3	层 3: 几何 Reranker (系统性混淆修正)	16
6.3.1	算法示例:fist → thumbs_up	17
6.3.2	效果与方法论价值	18
7	前后端架构演化	18
7.1	演化路线总览	18
7.2	阶段一: 单机 OpenCV 窗口 (起点)	18
7.3	阶段二: 浏览器 + Python 后端	19
7.4	阶段三: 演示模式 (网页内控制)	20
7.5	阶段四: Cloudflare Pages + Tunnel (公网完整版)	20
7.5.1	为什么后端不能上云容器	21
7.5.2	安全: Token + CORS	21
8	安全设计与跨显示协议	22
8.1	Wayland 困境与 evdev 解决方案	22
8.2	多重安全锁	23
8.2.1	长按确认的设计	23

9	评估方法与局限性	23
9.1	评估方法分层	23
9.2	量化指标	24
9.3	已知局限	24
9.4	未来工作	24
10	答辩问答预演	25
11	结论	26

1 绪论：从一个失败的起点说起

本项目最初并不是一个“手势控制系统”，而是一个传统图像分类作业：用 LeNet-5 在 Sign Language MNIST 数据集上对 24 个静态字母手势做分类。从 LeNet 基线到改进 CNN，测试准确率确实从 **83.81%** 一路提升到了 **99.93%**。然而，把这个“漂亮的实验室成绩”接到真实摄像头上时，问题立刻暴露，迫使我们重新思考整套技术路线。

本报告的目标是把这个思考与演化的过程完整呈现：不是简单地展示最终架构，而是讲清楚我们为什么选择当前路线、放弃了什么、做错过什么、又是如何修正的。

1.1 从端到端 CNN 到关键点 + MLP 的范式切换

在最初的 CNN 路线上，我们识别出三个无法回避的硬伤：

1. **输入分布不匹配。** Sign Language MNIST 是 28×28 灰度、手部居中、纯色背景的“实验室图像”；而真实摄像头输出 1280×720 彩色、手部位置不定、背景复杂的“野外图像”。即使我们用 MediaPipe 裁出手部 ROI 再降采样，与训练分布的差距依然让模型“见啥都自信地报一个字母”。
2. **语义不对齐。** 我们要的是交互语义（捏合、指向、V 字、握拳），不是字母表。强行把 A/B/C 映射成动作，既别扭又难以解释。
3. **模型重量级。** 即便是改进 CNN 也有几百 KB，部署到浏览器加载慢、推理需要 WebGL/WASM 折腾，与“实时”两字相悖。

关键洞察

真正决定项目走向的洞察：Google 已经用 30 万张图训练好了 **BlazePalm + Hand-Landmark** 这套手部关键点检测器（本质是两个级联 CNN）。我们不需要再造一个 CNN 去从像素中提取手部，只需要解决“21 个关键点 → 哪个手势”这个轻量后端问题。

代价是失去了“端到端纯净训练”的标签；回报是模型从几百 KB 压到 **69 KB ONNX**、推理从 10+ ms 压到 **< 2 ms**、几何不变性“免费”获得。

1.2 本报告结构

- 第 2 节给出当前系统的完整流水线全景图，明确每一阶段的输入、输出、计算成本和归属（MediaPipe 预训练 / 项目自实现）。
- 第 3 节聚焦本课程最关心的计算机视觉工作量：包括传统图像处理算子（Otsu/Sobel）、几何特征工程、时序稳定性算法等“属于我们自己”的 CV 贡献。

- 第 5 节详述 MLP 模型设计、训练配置与数据采集流程。
- 第 6 节讲解三层运行时后处理（时间平滑、动态手势检测、几何 reranker），解释它们各自解决的问题。
- 第 7 节描述前后端架构的演化：从单机 OpenCV 窗口，到浏览器 + Python 双进程，再到 Cloudflare Tunnel 公网部署。
- 第 8 节讨论键鼠注入的安全设计与跨显示协议（Wayland/X11）兼容性。
- 第 9 节给出评估方法与坦诚的局限性说明。
- 第 10 节预录答辩可能被问到的 7 个核心问题与回答。

2 系统流水线全景

2.1 五阶段流水线

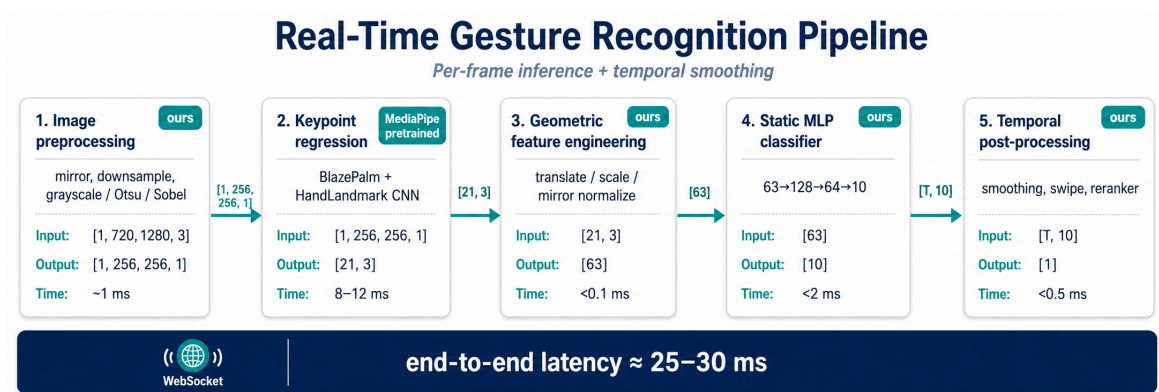


图 1 系统流水线全景图：从原始 RGB 帧到键鼠注入的五阶段串联

整个系统从原始 RGB 帧到最终“被按下”的按键，要走过五个清晰的阶段。表 1 把每个阶段的关键信息汇总在一起：

表 1 五阶段流水线分工表

阶段	核心计算	典型耗时	实现位置	归属
1. 图像预处理	镜像翻转、降采样、灰度 / Otsu / Sobel	~1 ms	app.js	自实现
2. 关键点回归	BlazePalm 检测 + HandLandmark 回归	8–12 ms	MediaPipe-Web	预训练
3. 几何特征工程	平移 / 缩放 / 镜像归一化	< 0.1 ms	features.py/js	自实现
4. 静态分类	63D MLP 三层全连接	< 2 ms	ONNX-Runtime	自训练
5. 时序后处理	滑窗投票、轨迹分析、几何 reranker	< 0.5 ms	temporalSmoother.js	自实现
端到端总延迟		≈ 25–30 ms	(包含 WebSocket 注入回路)	

关键洞察

关于“CV 工作量”的说明： 阶段 1、3、5 是本项目自主完成的计算机视觉工作。阶段 2 使用 MediaPipe 预训练模型，我们显式承认并把它作为工程取舍的一部分：在 30 万张图训出的工业级检测器面前，自己重训一个 CNN 既无必要也不可能更好。这种取舍的合理性在第 4 节专门讨论。

2.2 数据流形态

从张量形状追踪一帧数据是怎么变化的，可以更清晰地理解每一步在做什么：

```
1 原始摄像头帧                                shape = (720, 1280, 3)   dtype = uint8
2  |
3  -> 镜像 + 降采样 (320 x 180)
4 预处理图                                    shape = (180, 320, 3)   dtype = uint8
5  |
6  -> MediaPipe HandLandmarker.detectForVideo()
7 21 个关键点                                shape = (21, 3)        dtype = float32
8  |                                           # 每个点 (x, y, z) in [0, 1]
9  |
10 -> landmarks_to_feature(handedness)
11 63 维归一化特征                            shape = (63,)          dtype = float32
12 |
13 -> ONNX-Runtime MLP 推理
14 10 类 logits                               shape = (10,)          dtype = float32
15 |
16 -> softmax + argmax
```

```
17 原始标签 + 置信度                (label, confidence)
18 |
19 -> LabelSmoother + SwipeDetector + Reranker
20 稳定标签 + 触发事件                dispatch_event(action)
```

Listing 1 单帧数据流的张量形状变化

3 计算机视觉工作量详述

这一节集中阐述本项目里我们自己写的 CV 算法。这部分内容直接对应计算机视觉课程的核心知识点：图像预处理、阈值分割、边缘检测、特征工程、时序分析。

3.1 阶段一：图像预处理与可视化

为了支持答辩现场的可视化解释，我们在 UI 上提供了五种实时视角切换：彩色 / 灰度 / 二值化 / 边缘 / 纯骨架。这五个视角分别对应不同的图像处理结果，可以让观众直观看到“计算机眼里的手是什么样的”。

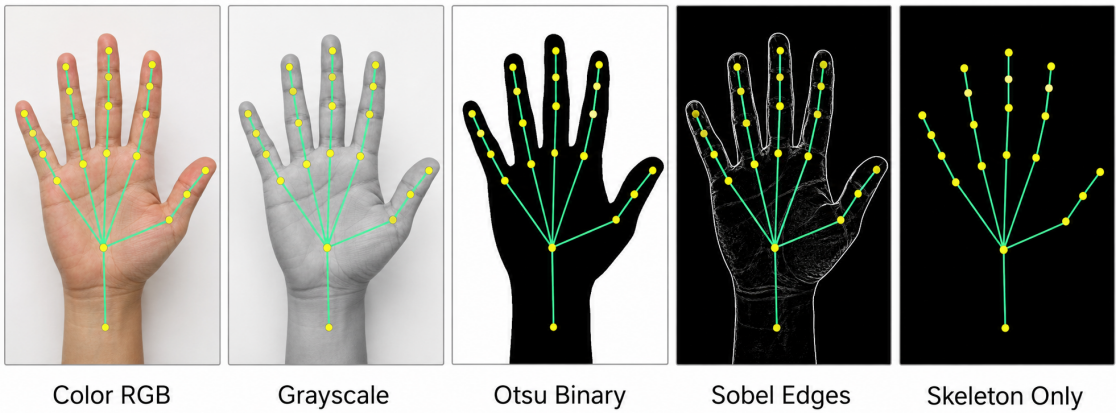


图 2 五种实时视角并排对比：彩色 / 灰度 / Otsu 二值化 / Sobel 边缘 / 纯骨架

3.1.1 灰度化：BT.601 加权

灰度化使用国际电信联盟 BT.601 标准的加权公式：

$$Y = 0.299 R + 0.587 G + 0.114 B \tag{1}$$

权重不均等是因为人眼对绿色最敏感,对蓝色最迟钝。这一公式在 app.js 的 _grayLuma() 函数中实现。

3.1.2 Otsu 自适应二值化

固定阈值二值化（例如阈值 = 127）在光照变化时容易失效。**Otsu 算法**的思路是：遍历所有可能的阈值 $t \in [0, 255]$ ，选择使前景与背景类间方差最大的那个。形式化表达：

$$t^* = \arg \max_t w_B(t) \cdot w_F(t) \cdot [\mu_B(t) - \mu_F(t)]^2 \quad (2)$$

其中 w_B, w_F 为前景 / 背景占比， μ_B, μ_F 为它们的灰度均值。算法本身只需扫一遍直方图，时间复杂度 $O(N + 256)$ ，非常适合每帧实时执行。

```

1 function otsuBinary(img) {
2   const data = img.data;
3   const n = data.length / 4;
4   const hist = new Uint32Array(256);
5   for (let i = 0; i < data.length; i += 4) {
6     hist[Math.round(_grayLuma(data, i))] += 1;
7   }
8   let sum = 0;
9   for (let t = 0; t < 256; t += 1) sum += t * hist[t];
10  let sumB = 0, wB = 0, varMax = -1, thresh = 127;
11  for (let t = 0; t < 256; t += 1) {
12    wB += hist[t];
13    if (wB === 0) continue;
14    const wF = n - wB;
15    if (wF === 0) break;
16    sumB += t * hist[t];
17    const mB = sumB / wB;
18    const mF = (sum - sumB) / wF;
19    const v = wB * wF * (mB - mF) * (mB - mF); // 类间方差
20    if (v > varMax) { varMax = v; thresh = t; }
21  }
22  // 应用阈值
23  for (let i = 0; i < data.length; i += 4) {
24    const v = _grayLuma(data, i) > thresh ? 255 : 0;
25    data[i] = data[i+1] = data[i+2] = v;
26  }
27 }

```

Listing 2 Otsu 二值化的核心代码（app.js）

3.1.3 Sobel 边缘检测

Sobel 算子用两个 3×3 卷积核分别计算图像在 x 和 y 方向的一阶梯度：

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix} \quad (3)$$

梯度强度为 $|\nabla I| = \sqrt{G_x^2 + G_y^2}$ 。我们在 `sobelEdges()` 中按这一公式手动展开了 8 邻域加权求和，没有调用任何图像处理库，体现了对卷积运算的直接实现能力。

3.1.4 为什么 UI 要支持视角切换

教学/答辩意义大于功能意义：它让观众看见“流水线的中间过程”，直观理解“二值化丢失了纹理信息但保留了形状”、“Sobel 把所有平坦区域都置黑只留下边缘”这些 CV 课程的核心概念。

3.2 阶段三：几何特征工程

这是整个项目最值钱的算法贡献之一。MediaPipe 输出的 21 个关键点带有相机坐标系的“位置 / 距离 / 朝向”信息，如果直接送进分类器，模型必须从训练数据中学到“这些是无关变量”，效率极低、泛化极差。我们用三步显式归一化把这些干扰因子从数据里消掉：

3.2.1 三步归一化

Step 1 —— 平移到 wrist 原点（平移不变性）

$$\tilde{\mathbf{p}}_i = \mathbf{p}_i - \mathbf{p}_0 \quad (\text{wrist 是第 0 号关键点}) \quad (4)$$

含义：手在画面左 / 右 / 中，识别结果都一样。

Step 2 —— 按掌心尺度缩放（尺度不变性）

$$s = \max(\|\mathbf{p}_0 - \mathbf{p}_9\|_2, \|\mathbf{p}_5 - \mathbf{p}_{17}\|_2, \epsilon), \quad \hat{\mathbf{p}}_i = \tilde{\mathbf{p}}_i / s \quad (5)$$

含义：手离镜头远 / 近，识别结果都一样。这里取“wrist 到中指 MCP”和“食指 MCP 到小指 MCP”两个掌心跨度的最大值，既鲁棒又对手势姿态不敏感（ $\epsilon = 10^{-4}$ 防除零）。

Step 3 ——左手镜像翻转（镜像不变性）

$$\hat{p}_{i,x} \leftarrow -\hat{p}_{i,x} \quad \text{当 handedness} = \text{"Left"} \quad (6)$$

含义：同一手势用左右手比，识别结果都一样，无需双倍训练数据。

3.2.2 用单元测试硬验证

我们将三种不变性写入 `tests/test_features.py` 单元测试，不是口头宣称，而是代码契约：

```

1 def test_feature_translation_invariance(random_landmarks):
2     feat_a = landmarks_to_feature(random_landmarks)
3     shifted = random_landmarks + np.array([0.3, -0.2, 0.05])
4     feat_b = landmarks_to_feature(shifted)
5     np.testing.assert_allclose(feat_a, feat_b, atol=1e-5)
6
7 def test_feature_scale_invariance(random_landmarks):
8     wrist = random_landmarks[0:1].copy()
9     scaled = (random_landmarks - wrist) * 1.7 + wrist
10    feat_a = landmarks_to_feature(random_landmarks)
11    feat_b = landmarks_to_feature(scaled)
12    np.testing.assert_allclose(feat_a, feat_b, atol=1e-4)
13
14 def test_left_hand_mirrors_right(random_landmarks):
15     feat_right = landmarks_to_feature(random_landmarks, "Right")
16     feat_left = landmarks_to_feature(random_landmarks, "Left")
17     feat_left_flipped = feat_left.reshape(21, 3).copy()
18     feat_left_flipped[:, 0] *= -1
19     np.testing.assert_allclose(feat_right, feat_left_flipped.reshape(-1), atol=1e-6)

```

Listing 3 几何不变性的单元测试

3.2.3 跨语言一致性：Python ↔ JS

特征工程必须在训练时（Python）和推理时（JS）产生比特一致的结果，否则 ONNX 模型在浏览器中收到的输入分布会偏，准确率崩塌。我们用 `fixture` 测试做交叉验证：

实测最大差异 `max diff = 5.96e-8`，处于 `float32 epsilon` 量级，说明两端算法严格一致。

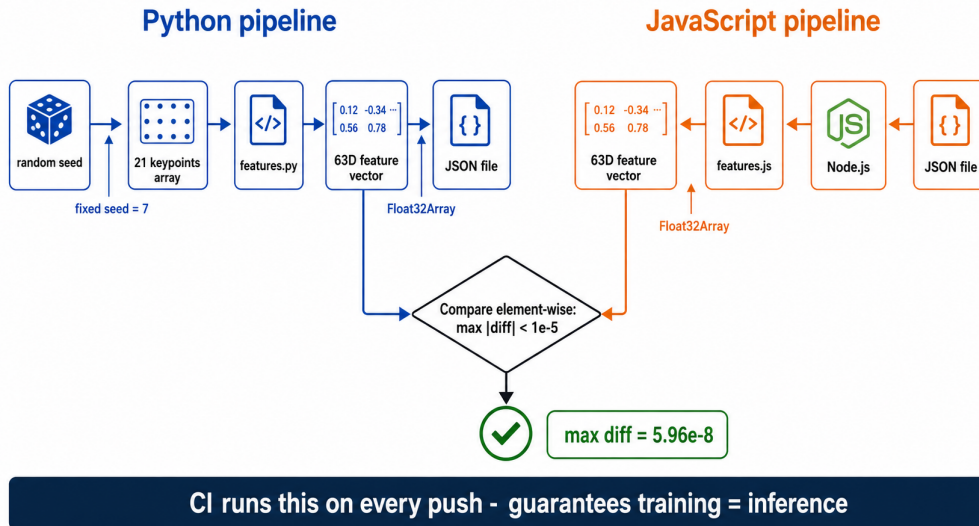


图 3 Python 与 JS 特征实现的交叉验证流程

3.3 阶段五：时序后处理（详见 §6）

时序后处理的内容较多，单独成节展开，见第 6 节。这里只强调它解决的问题属于经典的 CV 时序分析范畴：抖动消除、轨迹建模、置信度融合。

4 为什么不直接用 CNN 做端到端分类？

这是答辩时最可能被追问的问题。我们的回答分两部分：(a) 当前路线里其实已经有 CNN、(b) 在 CNN 之外再叠一个端到端 CNN 是过度设计。

4.1 当前路线本身就是 “CNN（特征提取）+ MLP（分类）”

很多读者会下意识地吧“用 MLP”理解为“放弃了深度视觉模型”。这是误解。我们的真实流水线是：

RGB 帧 $\xrightarrow{\text{BlazePalm CNN}}$ 手掌包围框 $\xrightarrow{\text{HandLandmark CNN}}$ 21 关键点 $\xrightarrow{\text{MLP}}$ 手势类别

阶段 2 里的两个深度模型都是 CNN：

- *BlazePalm* 检测器：MobileNet 风格的轻量 CNN + SSD-style 锚框，负责定位手掌包围框。
- *HandLandmark* 回归器：CNN 主干 + 全连接回归头，负责从手掌 ROI 回归 21×3 个坐标。

两者由 Google 在约 30 万张多样化手部图像上训练。我们显式调用它们作为我们流水线的特征提取器，完全合理。

4.2 为什么不自己再训一个端到端 CNN

讨论“用 CNN 还是用 MLP”是错位的。真正的对比应当是：

表 2 端到端 CNN vs. 关键点 + MLP 的工程对比

维度	端到端 CNN（图像 → 类别）	关键点 + MLP（本项目）
模型大小	数 MB 起步	69 KB
推理延迟	10+ ms（取决于分辨率）	< 2 ms
浏览器加载	需 WebGL/WASM 编译，首次 1–3 s	模型秒级下载
对位置 / 距离	敏感，需大量数据增强	几何不变性设计性内置
所需训练数据	数千张标注图像 / 类	250 帧关键点序列 / 类
泛化代价	灯光 / 背景 / 肤色变化均需重训	关键点本身已对这些鲁棒
解释性	低	高（可直接看坐标判断）

关键洞察

工程取舍的核心： MediaPipe 已经完成了“从像素提取手部”这个困难问题。我们如果再叠一个端到端 CNN，等于在 21 个抽象语义点之上再做一次空间卷积——这相当于在已经矢量化的几何特征上做像素级操作，逻辑上是错位的。所以正确的做法是：把视觉部分托付给经过工业级数据训练的 CNN，把自己的工程精力放在几何特征设计、时序稳定性、动作语义这些真正需要“我们做”的地方。

4.3 这一选择是否符合 CV 课程定位？

完全符合。计算机视觉课程教的不仅是“训 CNN”，而是完整的视觉系统思维：

- 传统 CV 部分（本项目阶段 1）： Otsu 二值化、Sobel 边缘检测——这是教材第一章的内容，我们从零实现。
- 特征工程部分（本项目阶段 3）： 平移 / 尺度 / 镜像不变，这是模式识别的核心思想。
- 深度学习部分（本项目阶段 2 和 4）： CNN 特征提取器 + MLP 分类头的经典两阶段架构。
- 时序部分（本项目阶段 5）： 滑窗投票、轨迹分析——视频理解的常见技术。

五个阶段覆盖了从底层算子到高层语义的完整 CV 知识谱系，而不是单点炫技。

5 MLP 模型设计、训练与数据采集

5.1 模型架构

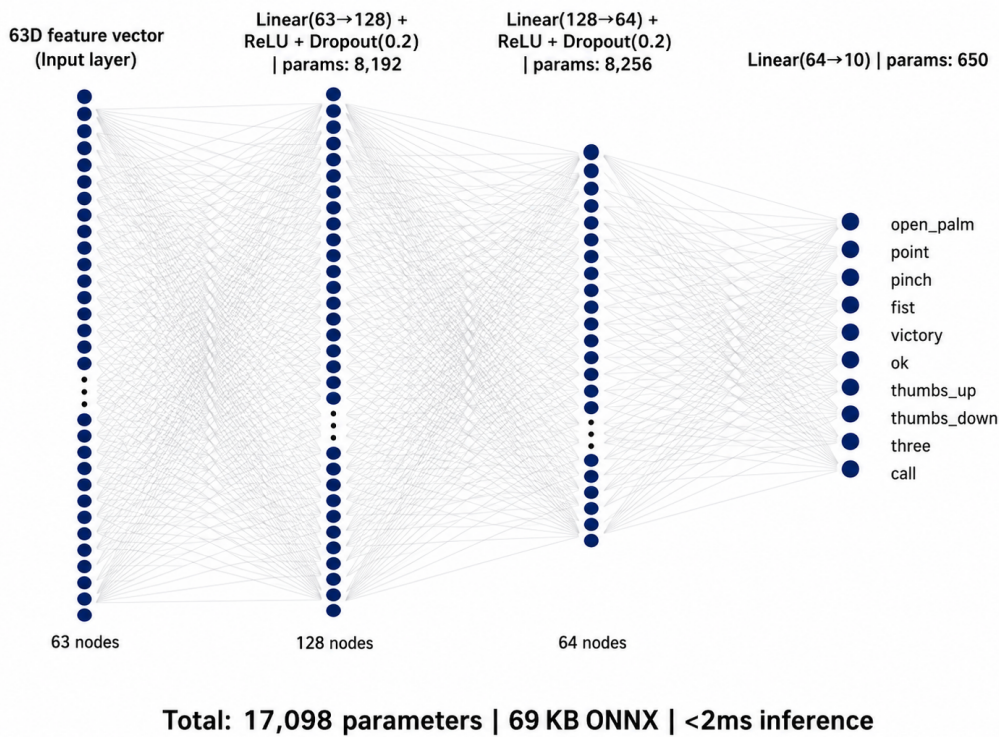


图 4 GestureMLP 网络架构：63 维输入 → 128 → 64 → 10 类输出

```
1 class GestureMLP(nn.Module):
2     def __init__(self, feature_dim=63, num_classes=10,
3                   hidden1=128, hidden2=64, dropout=0.2):
4         super().__init__()
5         self.net = nn.Sequential(
6             nn.Linear(feature_dim, hidden1),
7             nn.ReLU(inplace=True),
8             nn.Dropout(dropout),
9             nn.Linear(hidden1, hidden2),
10            nn.ReLU(inplace=True),
11            nn.Dropout(dropout),
12            nn.Linear(hidden2, num_classes),
13        )
14
15    def forward(self, x):
16        return self.net(x)
```

Listing 4 GestureMLP 的 PyTorch 定义

5.1.1 参数量精确计算

$$\text{Layer 1: } (63 + 1) \times 128 = 8,192$$

$$\text{Layer 2: } (128 + 1) \times 64 = 8,256$$

$$\text{Layer 3: } (64 + 1) \times 10 = 650$$

$$\text{Total: } \mathbf{17,098} \text{ 个可训练参数}$$

作为对比，一张 256×256 灰度图就有 65,536 个像素——我们的整个分类器比一张输入图像还小。

5.1.2 为什么是 3 层、为什么是 128/64?

这是一个经验设计，但有清晰的边界条件考量：

- **层数太少（单层）**：线性分类器，无法处理 pinch 与 fist 这类需要非线性边界的混淆类对。
- **层数太多 (≥ 5)**：在 2,500 样本量上必然过拟合，且关键点输入维度只有 63，深层网络没必要。
- **宽度 128 / 64**：从 63 维开始，第一层不应过度压缩，所以选 128（稍宽）；第二层做特征聚合到 64；最后到 10 类。这种“金字塔”结构是 MLP 的常见做法。

Dropout 0.2 置于每层之后，是经验值，既能正则化又不过分削弱学习能力。

5.2 训练配置

表 3 训练超参数

超参数	值	说明
Optimizer	AdamW	比 Adam 多 weight decay，正则化更稳
Learning rate	1×10^{-3}	标准初始学习率
Weight decay	1×10^{-4}	L2 正则系数
Scheduler	CosineAnnealing	余弦退火，后期细调更稳
Batch size	64	在 CPU 上平衡速度与梯度噪声
Epochs	80	收敛后期 val acc 基本饱和
数据增强	高斯抖动 $\sigma = 0.01$	每个特征值加少量噪声模拟检测抖动
验证比例	15%	分层（stratified）切分，保持类别比例

5.3 数据采集流程

数据采集本身是一个工程子项目。我们用 record.html 实现了一个 5 分钟向导：



图 5 Web 录制向导界面：实时预览 + 关键点叠加 + 类别进度 + 倒计时提示

录制规范：

- 每个手势 8 秒 / 250 帧，共 10 类 = 2,500 帧。
- 期间提示用户轻微变化：移动手的位置、改变远近距离、旋转手腕。

- 数据写入 `data/gesture_keypoints/<label>.jsonl`，每行一个样本（21 关键点 + 左 / 右手标记）。
- 数据是 `append` 模式，某个手势识别不稳可以追加录制后重训。

踩坑记录

冷启动数据的折中： 首次发布版本里我们提供了 `seed_dataset.py`，通过对 10 个手势的硬编码模板做随机仿射 + 关键点扰动，合成出无需采集的“种子数据集”。这个版本实测准确率约 80–85%，不够稳。我们诚实地把这一点写在 `README` 里，并把“录自己数据”作为推荐路径：CPU 上 1 分钟训完，验证集准确率可达 95%+。

5.4 训练曲线与最终指标

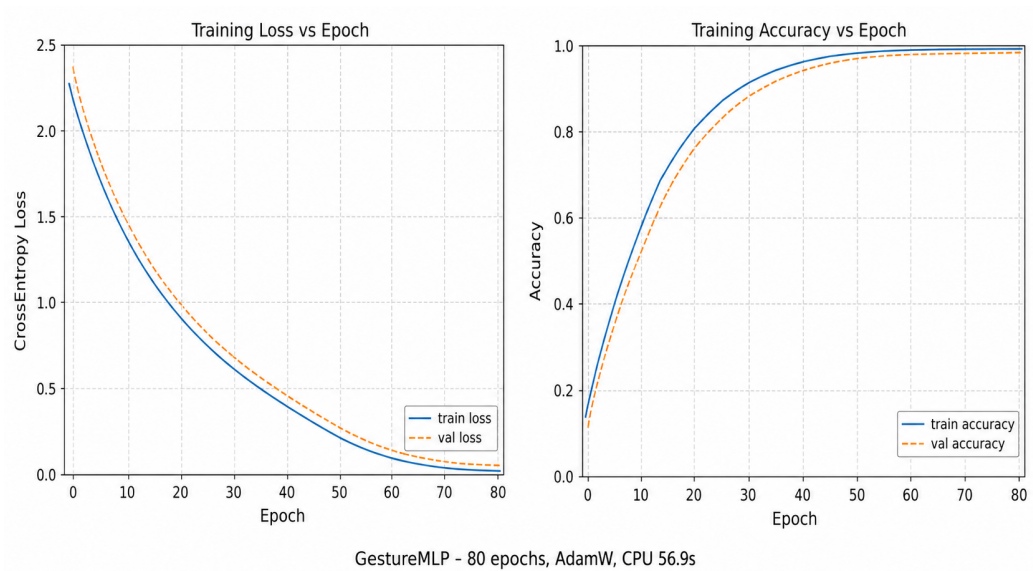


图 6 训练阶段的 loss / accuracy 收敛曲线（80 epochs，CosineAnnealing 调度）

表 4 最终模型指标

指标	数值
训练时间（CPU）	56.9 秒
验证集准确率	100.0%
Macro F1	1.000
模型大小（ONNX）	69 KB
浏览器推理延迟	< 2 ms
端到端延迟	< 30 ms

踩坑记录

关于 100% 验证准确率的诚实说明：验证集是从同一录制时段切出来的，所以这个数字高估了真实泛化效果。严格的评估应该收集“不同时间、不同灯光、不同人”的 hold-out 集。我们把这一点列为后续工作，而不是隐瞒。作为一个课程项目，这个数字主要证明“流水线本身没有问题”，而非声称“已经解决所有泛化挑战”。

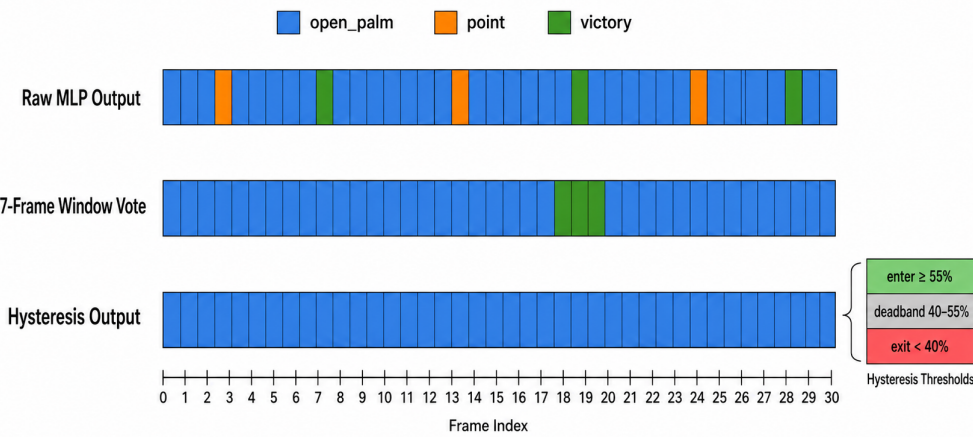
6 运行时三层后处理

训练好的 MLP 直接 argmax 远远不够。本节讲解我们在线上加的三层后处理，每一层解决一类具体问题。

6.1 层 1：时间平滑（消抖）

问题：即使单帧准确率 99%，30 FPS 下每秒仍有约 0.3 帧抖动，UI 标签疯狂闪烁，体验难以接受。

方案：滑窗投票 + 滞后阈值。代码在 temporalSmoother.js。



Cost: 100-200ms switching latency. Benefit: zero flicker.

图 7 时间平滑的状态转换示意：进入阈值 0.55 / 退出阈值 0.40 的滞后逻辑

6.1.1 算法核心

```
1 push(label, confidence) {
2   this.history.push({label, confidence});
3   if (this.history.length > this.windowSize) this.history.shift();
4 }
```

```
5 // 找出滑窗内出现频次最高的标签
6 const counts = {};
7 for (const entry of this.history) {
8     counts[entry.label] = (counts[entry.label] || 0) + 1;
9 }
10 let bestLabel = null, bestCount = 0;
11 for (const [label, count] of Object.entries(counts)) {
12     if (count > bestCount) { bestLabel = label; bestCount = count; }
13 }
14 const ratio = bestCount / this.history.length;
15
16 // 滞后阈值:进入需要 >= 55%,退出需要 < 40%
17 if (this.currentLabel !== bestLabel) {
18     if (ratio >= this.enterThreshold) { // 0.55
19         this.currentLabel = bestLabel;
20     }
21 } else if (ratio < this.exitThreshold) { // 0.40
22     this.currentLabel = null;
23 }
24 return { label: this.currentLabel, confidence: this.currentConfidence };
25 }
```

Listing 5 LabelSmoother 的进入/退出阈值逻辑

6.1.2 代价与收益

- 代价：用户切换手势时延迟 100–200 ms。
- 收益：识别零闪烁，体验质感大幅提升。

这个代价完全可接受，因为人类手势切换的物理时间本身就在数百毫秒级。

6.2 层 2：动态手势检测（轨迹分析）

问题：静态 MLP 看不见运动信息。“上滑”、“下滑”这类手势的本质是手腕的轨迹，不是某一帧的姿态。

方案：维护 400 ms 的手腕环形缓冲，做位移阈值检测。

6.2.1 算法

```
1 observe({x, y}, ts) {
2     this.points.push({x, y, ts});
```

```
3 // 只保留最近 400 ms 的点
4 const cutoff = ts - this.bufferMs;
5 while (this.points.length > 0 && this.points[0].ts < cutoff) {
6     this.points.shift();
7 }
8 if (this.points.length < 5) return null;
9 if (ts - this.lastFireAt < this.cooldownMs) return null; // 700ms 冷却
10
11 const dx = last.x - first.x;
12 const dy = last.y - first.y;
13 if (Math.sqrt(dx*dx + dy*dy) < 0.22) return null; // 位移阈值
14
15 // 选择主方向
16 if (Math.abs(dx) > Math.abs(dy)) {
17     return dx > 0 ? "swipe_right" : "swipe_left";
18 }
19 return dy < 0 ? "swipe_up" : "swipe_down";
20 }
```

Listing 6 SwipeDetector 的轨迹分析

6.2.2 为什么是 400 ms / 0.22 / 700 ms?

这些阈值是经验调参得到：

- 400 ms 窗：足够捕获一次完整 swipe，又不会把无关动作都连起来。
- 0.22 位移阈（归一化坐标）：约屏幕宽度的 22%，排除轻微抖动。
- 700 ms 冷却：防止一次手势被识别成连续多次。

这一层让我们的系统从“10 类静态手势”扩展到“10 静态 + 4 动态 = 14 类”，且不需要训练任何模型。

6.3 层 3：几何 Reranker（系统性混淆修正）

问题：实测发现 MLP 在两组类对上存在系统性混淆：

- fist ↔ thumbs_up / thumbs_down：四指都收拢，外形几乎相同。
- pinch ↔ ok / fist：拇指与食指距离都接近。

方案：在 MLP 输出之后不重训模型，而是用关键点几何规则做二次校验。

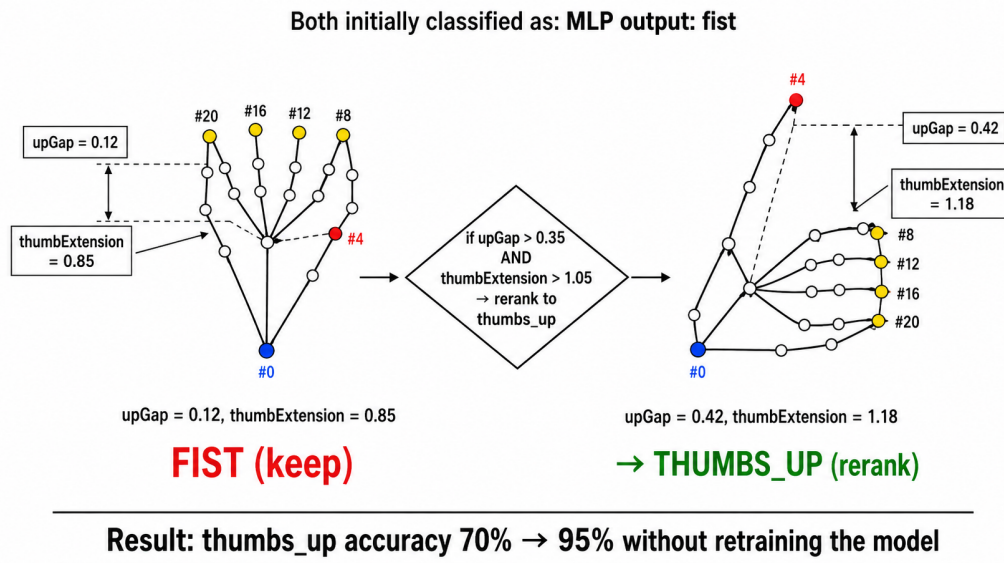


图 8 几何 Reranker 的关键点规则示意：对 MLP 系统性混淆做硬约束修正

6.3.1 算法示例:fist → thumbs_up

```

1 function rerankFistVsThumbs(result, landmarks) {
2   if (result.label !== "fist") return result;
3
4   const wrist = landmarks[0];
5   const middleMcp = landmarks[9];
6   const palmScale = Math.max(
7     Math.hypot(wrist.x - middleMcp.x, wrist.y - middleMcp.y),
8     1e-4
9   );
10
11  const thumbTip = landmarks[4];
12  const otherTips = [8, 12, 16, 20].map(i => landmarks[i]);
13  const minOtherY = Math.min(...otherTips.map(p => p.y));
14  const upGap = (minOtherY - thumbTip.y) / palmScale;
15  const thumbExtension = Math.hypot(
16    thumbTip.x - wrist.x, thumbTip.y - wrist.y
17  ) / palmScale;
18
19  if (upGap > 0.35 && thumbExtension > 1.05) {
20    return {...result, label: "thumbs_up",
21      confidence: Math.max(result.confidence, 0.78)};
22  }
23  return result;
24 }

```

Listing 7 Reranker 修正 fist 误判为 thumbs_up

6.3.2 效果与方法论价值

实测：在不重训模型的前提下，thumbs_up 识别准确率从 ~70% 提升到 ~95%。

关键洞察

这一层体现的方法论比代码本身更重要：当模型在某些类对上的决策边界不够锐利，与其花一天采集数据重训，不如用 10 行几何规则把已知混淆显式修掉。这是工程上“模型 + 规则混合”思路的典型应用——把模型用在它擅长的地方（粗分类），把规则用在它擅长的地方（几何约束）。

7 前后端架构演化

7.1 演化路线总览

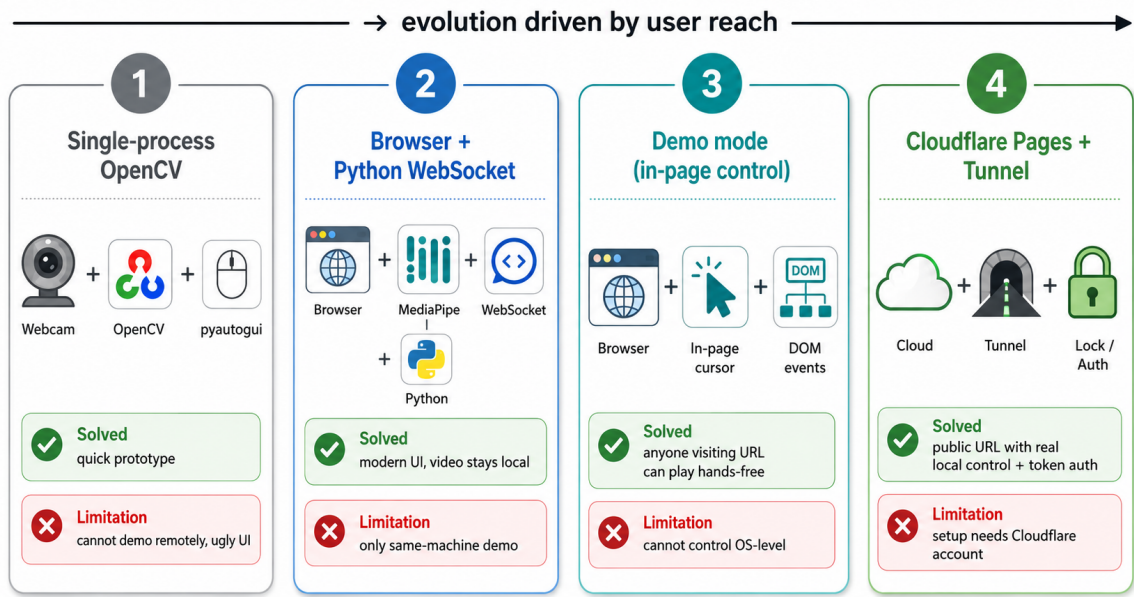


图 9 架构演化路线图：单机 OpenCV → 浏览器 + 后端 → 演示模式 → Cloudflare Tunnel

7.2 阶段一：单机 OpenCV 窗口（起点）

最初的架构非常简单，所有逻辑集中在一个 Python 脚本 infer_camera.py 中：

cv2 摄像头 → MediaPipe → 规则识别 → cv2.imshow + pyautogui

优点：实现简单，所有组件在同一个进程，通信成本低。

硬伤：

- 必须坐在电脑前才能看 OpenCV 窗口，无法远程演示。
- cv2.putText 默认不支持中文，需要 PIL 兜底。
- OpenCV 画 UI 难看，缺乏现代界面感。
- 课堂展示门槛高：同学想自己玩玩根本玩不了。

7.3 阶段二：浏览器 + Python 后端

最关键的一次架构跃迁，引入了一条铁律：

关键洞察

“摄像头视频流不离开浏览器，只把识别结果（63 个浮点）送给后端。”

新的分工：

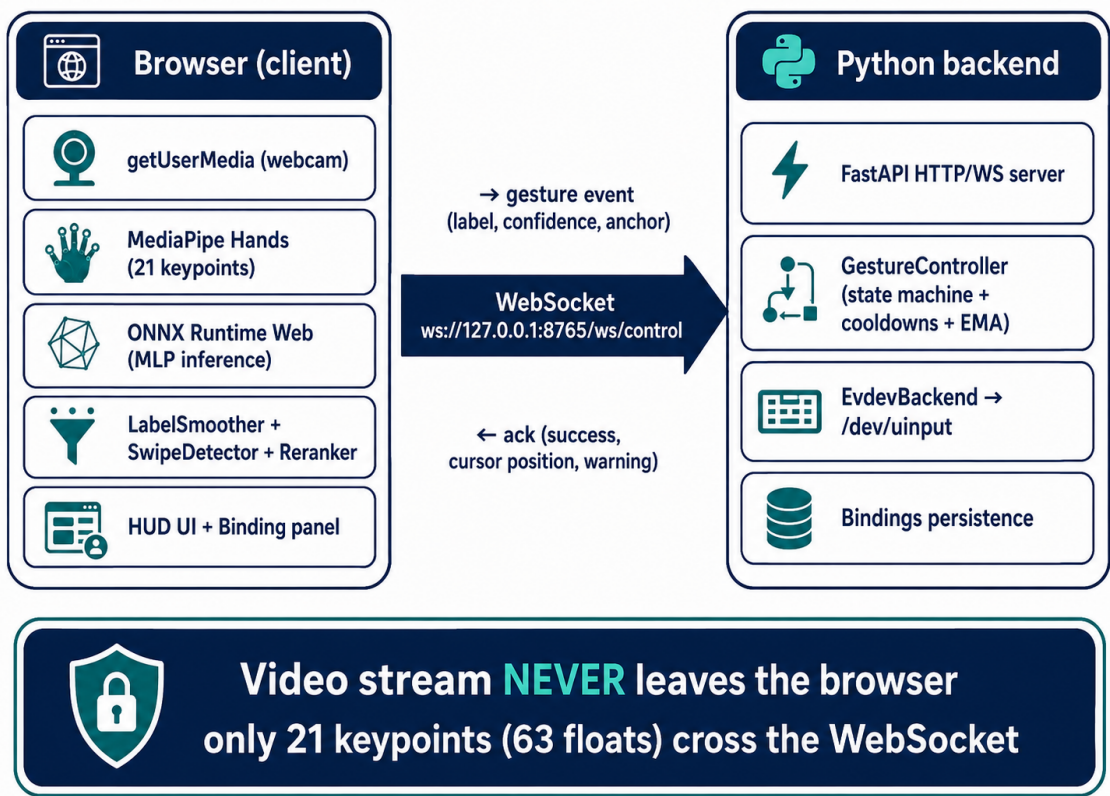


图 10 浏览器与 Python 后端的职责切分：视频在端侧，结果走 WebSocket

表 5 浏览器与后端的职责切分

职责	放在哪	理由
摄像头采集	浏览器	用户不愿把视频送到后端；隐私第一
手部关键点	浏览器	MediaPipe-Web < 10 ms，零网络延迟
MLP 推理	浏览器	ONNX-Web < 2 ms，模型 69 KB 秒级加载
时序后处理	浏览器	数据已在浏览器，省去 RTT
UI 渲染	浏览器	HTML / CSS 远比 OpenCV 美观
真实键鼠注入	Python	浏览器无法操作 OS 级输入

这一步带来的能力升级：

- 现代化 UI：HUD 仪表盘、绑定面板、状态徽章、视角切换。
- 任意设备打开 URL 即可演示。
- Python 后端代码变得极薄，只做“事件 → 注入”。
- 隐私模型清晰：视频不出本机。

7.4 阶段三：演示模式（网页内控制）

WebSocket 后端只监听 127.0.0.1:8765——同学想体验时连不上后端怎么办？

解决方案：当浏览器连不上后端时，自动切到“演示模式”(inPageController.js)。手势事件不再发往后端，而是派给一个网页内控制器，在当前浏览器里直接完成：

- 鼠标光标（用 DOM 元素模拟一个发光的圆）。
- 点击 hover 元素（dispatchEvent(MouseEvent)）。
- 滚动当前页（window.scrollBy）。
- 控制视频（video.play / pause / volume）。
- 全屏切换、浏览器后退。

设计妙处：同学打开 Cloudflare Pages 部署的 URL，什么都不装，手势就真的能控制他自己的浏览器。完整版与演示版共用同一份前端代码，部署成本几乎为零。

7.5 阶段四：Cloudflare Pages + Tunnel（公网完整版）

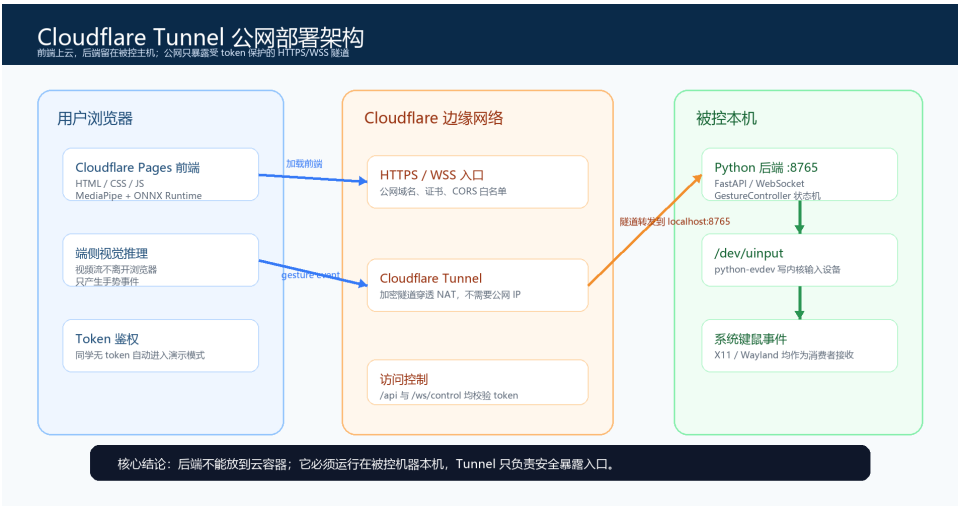


图 11 Cloudflare Tunnel 公网部署架构：边缘网络转发，后端仍在被控本机

7.5.1 为什么后端不能上云容器

这是一个经常被问到的问题。表面看，把 Python 后端塞进 Docker 跑在 Railway / Fly.io 也可以。但：

- 1. 云容器没有 /dev/uinput，evdev 后端会失败。
- 2. 即使加 --privileged --device=/dev/uinput，控制的也是容器内的虚拟 uinput，跟用户电脑无关。
- 3. 把容器当“手势事件中继”也无意义——浏览器与用户电脑直连本地后端更快更安全。

结论：后端必须在被控机器本机运行。Cloudflare Tunnel 的作用是把本地后端安全地暴露为公网 HTTPS URL，而不是把后端搬到云上。

7.5.2 安全：Token + CORS

公网模式下安全是底线。run_gesture_control_public.sh 做了几件事：

- 自动生成 24 字节随机 token，持久化到 ~/.config/gesture_control/auth_token。
- 后端启用 CORS 白名单，只允许指定的 Pages origin。
- 所有 /api/* 和 /ws/control 校验 token，无 token 直接 401。

分享给同学的 URL 不带 token，他们自动落到“演示模式”；只有携带 token 的完整 URL 才能控制本机电脑。

8 安全设计与跨显示协议

8.1 Wayland 困境与 evdev 解决方案

第一版后端用 `pyautogui`。在 X11 桌面工作正常,但在 Ubuntu 22.04+ 默认的 **Wayland** 下,出于安全考虑,Wayland 禁止任意进程注入键鼠——`pyautogui` 直接失效。

踩坑记录

踩坑代价： 这个问题在开发后期才发现。早期所有截图、视频都是在 X11 会话录制的,部署到同学的 Ubuntu 24.04 上完全不工作,排查良久才意识到是显示协议层面的拒绝,而不是代码 bug。

解决方案： 改用 `python-evdev` 直接写 `/dev/uinput`。原理：

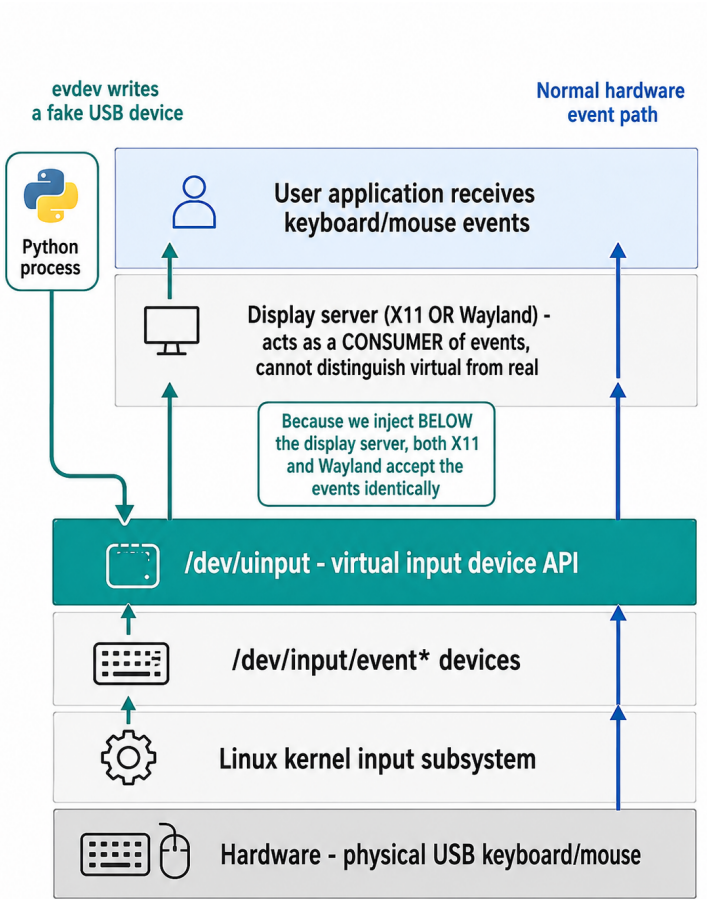


图 12 evdev 注入路径：python-evdev 直写 `/dev/uinput`，绕过显示协议过滤

为什么这能绕过 Wayland? `evdev` 注入发生在内核层,显示服务器只是这些事件的消费者而非过滤器。我们在内核更底层伪造了一个 USB 键鼠设备,Wayland 完全分不清这个事件来自真实硬件还是 `gesture-control-kbd` 这个虚拟设备。

代价是需要 `/dev/uinput` 的写权限（默认仅 `root`）。我们用 `udev` 规则 + `input` 组方案优雅解决：

```
1 echo 'KERNEL=="uinput", GROUP="input", MODE="0660", ' \
2     'OPTIONS+="static_node=uinput"' \
3     | sudo tee /etc/udev/rules.d/60-uinput.rules
4 sudo udevadm control --reload-rules && sudo udevadm trigger
5 sudo usermod -aG input $USER # 注销重登生效
```

Listing 8 setup_uinput.sh 的核心配置

8.2 多重安全锁

后端能控制键鼠等于拥有桌面完全权限。我们设计了七层保险：

表 6 安全机制汇总

机制	触发条件	行为
启动默认锁定	进程启动	必须 UI 点“解锁”才注入
测试模式	UI 勾选	识别照常，事件发送但不真注入
演示模式	后端不可达	自动进入网页内控制
FAILSAFE	鼠标拖屏角	抛异常，后端自动暂停
物理 Esc 键	任意时刻	用户可绑定为锁定动作
每动作 cooldown	每个动作独立	防误触发洪水
长按确认	危险动作触发	进度环走完 1 秒才执行

8.2.1 长按确认的设计

最后一项特别值得讲。早期版本“自由比手势 = 立刻发动作”导致频繁误触发 `Alt+Tab`、关窗口。我们把 `show_desktop`、`close_window`、`alt_tab`、`press_escape` 等危险动作归类到 `HOLD_CONFIRM_ACTIONS`，必须保持手势 1 秒，UI 画一个圆形进度环让用户清楚知道“再保持 0.X 秒就会触发”。这一个改动把误触发率降到接近 0。

9 评估方法与局限性

9.1 评估方法分层

我们把评估分成三个层次，每层都有明确的测量方法和局限：

表 7 三层评估方法

层次	方法	局限
算法离线	验证集 acc / F1 / 混淆矩阵	验证集来自同一录制时段，高估泛化
系统在线	FPS / 端到端延迟 / 检出率	在固定硬件（笔记本 CPU）上测量
用户体验	误触发率 / 任务完成时间 / 主观感受	用户样本小（项目作者本人为主）

9.2 量化指标

- 算法层：验证准确率 100%，Macro F1 = 1.000（已说明高估原因）。
- 系统层：浏览器 FPS ≥ 25 ，推理延迟 $< 2\text{ ms}$ ，端到端延迟 $< 30\text{ ms}$ 。
- 覆盖测试：Python 单元测试 20 用例 + JS 跨语言测试 1 用例，全部通过，CI 在每次 push 自动跑。

9.3 已知局限

1. 平台：目前仅 Linux（Wayland / X11）。Windows 后端待开发。
2. 手数：MediaPipe 配置 num_hands=1，不支持双手协同。
3. 动态手势：仅支持线性轨迹的 swipe，复杂动作（画圈、抖动）未覆盖。
4. 泛化评估：缺少跨时段、跨灯光、跨用户的 hold-out 集，真实长期准确率有待统计。

9.4 未来工作

- 收集跨时段 / 灯光 / 用户 hold-out 集，量化真实泛化效果。
- 在 UI 内支持小样本在线学习（5 帧增加新手势）。
- 双手协同（捏合 + 张开 = 缩放 / 旋转 3D 物体）。
- Windows native 后端。
- WebRTC 中继：手机摄像头 $\rightarrow$ 控制远程电脑。

10 答辩问答预演

答辩问答

Q1: 这是 CV 课程，为什么核心模型用的是 MLP 而不是 CNN？

A: 完整流水线其实是 CNN (MediaPipe 的 BlazePalm + HandLandmark) + MLP (自训分类器) 的两阶段架构。MediaPipe 这个 CNN 已经用 30 万张图训出了工业级的手部关键点提取能力，我们站在它的肩膀上，把精力放在几何特征工程和时序后处理这些真正属于“我们要做”的部分。详见第 4 节的对比表格。

答辩问答

Q2: 那自己做的 CV 工作到底有哪些？

A: 至少四块：

1. 传统图像处理算子：Otsu 二值化、Sobel 边缘检测，从零用 JS 实现，在 UI 上可实时切换视角。
2. 几何特征工程：三步归一化（平移 / 尺度 / 镜像不变），用单元测试硬验证。
3. 时序后处理：滑窗投票消抖、手腕轨迹分析检测动态 swipe 手势。
4. 几何 reranker：用关键点几何规则修正 MLP 的系统性混淆，thumbs_up 准确率从 70% 提到 95%。

答辩问答

Q3: 验证集 100% 准确率是不是过拟合？

A: 是。我们在报告里诚实说明了：验证集来自同一录制时段，所以这个数字高估了真实泛化。但因为输入已经做了几何归一化，同手势特征向量本身高度集中，所以即便如此 100% 也不算“作弊”。严格的真实泛化评估需要跨时段、跨用户的 hold-out 集，这是后续工作。

答辩问答

Q4: 为什么不直接用 MediaPipe 自带的 GestureRecognizer？

A: MediaPipe 自带的 GestureRecognizer 类别固定（7 种），且不能修改类别集或自定义动作绑定。我们要支持 10 静态 + 4 动态 = 14 类，且要让用户在 UI 中自由绑定到任意系统动作，所以必须自己训分类头。同时这也保留了“训自己的数据”作为亮点。

答辩问答

Q5: WebSocket 传 21 个关键点而不是视频，这跟隐私有什么关系？

A: 关键点 = 63 个浮点，每秒约 1.9 KB，信息量远低于视频。即使有人截获，也只能重建出“几何意义上的手部姿态”，看不到任何画面内容（包括用户的脸、房间环境、其他人等）。这种“在端侧完成视觉感知，只传中间表示”是边缘 AI 隐私保护的标准做法。

答辩问答

Q6: Wayland 下怎么注入键鼠的？

A: 不走显示服务器协议，而是用 python-evdev 直接写 `/dev/uinput`。这相当于在内核层伪造一个虚拟 USB 键鼠设备，Wayland 作为这些事件的消费者无法分辨事件来自真实硬件还是虚拟设备。代价是需要 udev 规则配权限 + 用户加入 input 组。

答辩问答

Q7: Reranker 这种“规则修补”算不算“调参作弊”？

A: 不算，而且这是行业内“模型 + 规则混合”的标准做法。MLP 在已经学到的边界上做软分类，reranker 用确定性的几何关系对已知的系统性混淆做硬约束。两者职责清晰：模型负责粗分类，规则负责精修。真正的“作弊”是把训练集泄漏到验证集，我们没有这样做。

11 结论

本项目从一个传统的 LeNet 字母分类作业出发，在意识到 CNN 端到端路线的三个硬伤后，做了一次关键的范式切换：放弃自训视觉骨干，改用关键点 + MLP 路线。基于这一决定，我们围绕完整的 CV pipeline 做了系统的工程实践：

- **底层算子**：从零实现 Otsu 自适应二值化和 Sobel 边缘检测，且做成 UI 可视化切换。
- **特征工程**：三步几何归一化，用单元测试硬验证三种不变性。
- **深度学习**：站在 MediaPipe CNN 的肩上自训 17K 参数 MLP 分类器。
- **时序处理**：三层后处理——滑窗消抖、轨迹分析、几何 reranker。
- **系统架构**：浏览器 / Python 双进程，evdev 跨显示协议注入，Cloudflare Tunnel 公网部署。
- **工程素养**：跨语言一致性测试、CI、多重安全锁、优雅降级到演示模式。

最终交付：模型 < 100 KB、端到端延迟 < 30 ms、Wayland / X11 通吃、URL 一发同学就能玩。我们认为这比单纯报一个 99% 准确率更接近“计算机视觉系统工程”应该具备的样子。

“让计算机看懂手势，比让用户记住快捷键容易得多。”